BEE 4750 Lab 2: Monte Carlo

Name:

ID:

> **Due Date**
>
> Thursday, 10/02/25, 9:00pm

Setup

The following code should go at the top of most Julia scripts; it will load the local package environment and install any needed packages. You will see this often and shouldn't need to touch it.

```
import Pkg
Pkg.activate(".")
Pkg.instantiate()
```

```
  Activating project at `~/Documents/GitHub/lab-2-nar84-design`
   Installed FillArrays          v1.14.0
   Installed Libmount_jll         v2.41.2+0
   Installed Xorg_libXfixes_jll      v6.0.2+0
   Installed Xorg_xcb_util_cursor_jll   v0.1.6+0
   Installed Libuuid_jll          v2.41.2+0
   Installed DataStructures        v0.19.1
   Installed Plots             v1.41.1
Precompiling project...
    616.9 ms   Libmount_jll
    619.2 ms   Rmath_jll
    651.1 ms   OpenSpecFun_jll
   1030.4 ms   FillArrays
    467.2 ms   Libuuid_jll
```

```
   483.7 ms    Xorg_libXfixes_jll
   486.1 ms    Xorg_xcb_util_cursor_jll
  1766.3 ms    DataStructures
   668.2 ms    FillArrays → FillArraysStatisticsExt
  1098.7 ms    Rmath
   698.9 ms    FillArrays → FillArraysSparseArraysExt
   589.6 ms    FillArrays → FillArraysPDMatsExt
   897.4 ms    Fontconfig_jll
   607.8 ms    SortingAlgorithms
   799.6 ms    QuadGK
  2466.4 ms    SpecialFunctions
  1042.9 ms    Cairo_jll
  3909.1 ms    ColorSchemes
  1471.6 ms    Qt6Base_jll
   524.0 ms    ColorVectorSpace → SpecialFunctionsExt
  1091.4 ms    HypergeometricFunctions
  1169.0 ms    HarfBuzz_jll
  2253.0 ms    StatsBase
  1466.0 ms    Qt6ShaderTools_jll
  1854.3 ms    StatsFuns
   901.0 ms    libass_jll
   852.8 ms    Pango_jll
  1411.8 ms    FFMPEG_jll
  1895.2 ms    Qt6Declarative_jll
  1175.9 ms    FFMPEG
  1543.9 ms    GR_jll
  4676.3 ms    Distributions
  7779.7 ms    PlotUtils
  1147.7 ms    Distributions → DistributionsTestExt
  3001.1 ms    GR
  1851.2 ms    PlotThemes
  2523.9 ms    RecipesPipeline
 36773.6 ms    Plots
 38 dependencies successfully precompiled in 52 seconds. 153 already precompiled.
```

```julia
using Random # random number generation
using Distributions # probability distributions and interface
using Statistics # basic statistical functions, including mean
using Plots # plotting
```

```
ArgumentError: ArgumentError: Package functions not found in current path.
- Run `import Pkg; Pkg.add("functions")` to install the functions package.
```

```
ArgumentError: Package functions not found in current path.
- Run `import Pkg; Pkg.add("functions")` to install the functions package.


Stacktrace:
  [1] macro expansion
    @ ./loading.jl:2296 [inlined]
  [2] macro expansion
    @ ./lock.jl:273 [inlined]
  [3] __require(into::Module, mod::Symbol)
    @ Base ./loading.jl:2271
  [4] #invoke_in_world#3
    @ ./essentials.jl:1089 [inlined]
  [5] invoke_in_world
    @ ./essentials.jl:1086 [inlined]
  [6] require(into::Module, mod::Symbol)
    @ Base ./loading.jl:2260
  [7] eval
    @ ./boot.jl:430 [inlined]
  [8] include_string(mapexpr::typeof(REPL.softscope), mod::Module, code::String, filename::St
    @ Base ./loading.jl:2734
  [9] #invokelatest#2
    @ ./essentials.jl:1055 [inlined]
```

```
[10] invokelatest
   @ ./essentials.jl:1052 [inlined]
[11] (::VSCodeServer.var"#217#218"{VSCodeServer.NotebookRunCellArguments, String})()
   @ VSCodeServer ~/.vscode/extensions/julialang.language-julia-1.149.2/scripts/packages/VS
[12] withpath(f::VSCodeServer.var"#217#218"{VSCodeServer.NotebookRunCellArguments, String},
   @ VSCodeServer ~/.vscode/extensions/julialang.language-julia-1.149.2/scripts/packages/VS
[13] notebook_runcell_request(conn::VSCodeServer.JSONRPC.JSONRPCEndpoint{Base.PipeEndpoint,
   @ VSCodeServer ~/.vscode/extensions/julialang.language-julia-1.149.2/scripts/packages/VS
[14] dispatch_msg(x::VSCodeServer.JSONRPC.JSONRPCEndpoint{Base.PipeEndpoint, Base.PipeEndpo
   @ VSCodeServer.JSONRPC ~/.vscode/extensions/julialang.language-julia-1.149.2/scripts/pac
[15] serve_notebook(pipename::String, debugger_pipename::String, outputchannel_logger::Base
   @ VSCodeServer ~/.vscode/extensions/julialang.language-julia-1.149.2/scripts/packages/VS
[16] top-level scope
   @ ~/.vscode/extensions/julialang.language-julia-1.149.2/scripts/notebook/notebook.jl:35
```

Overview

In this lab, we will conduct a Monte Carlo experiment and explore how sensitive Monte Carlo estimates are to the underlying probability distribution(s).

You should always start any computing with random numbers by setting a "seed," which controls the sequence of numbers which are generated (since these are not *really* random, just "pseudorandom"). In Julia, we do this with the `Random.seed!()` function.

```
Random.seed!(67)
```

```
TaskLocalRNG()
```

It doesn't matter what seed you set, though different seeds might result in slightly different values. But setting a seed means every time your notebook is run, the answer will be the same.

> **Seeds and Reproducing Solutions**
>
> If you don't re-run your code in the same order or if you re-run the same cell repeatedly, you will not get the same solution. If you're working on a specific problem, you might want to re-use `Random.seed()` near any block of code you want to re-evaluate repeatedly.

Probability Distributions and Julia

Julia provides a common interface for probability distributions with the `Distributions.jl` package. The basic workflow for sampling from a distribution is:

1. Set up the distribution. The specific syntax depends on the distribution and what parameters are required, but the general call is the similar. For a normal distribution or a uniform distribution, the syntax is

```
# you don't have to name this "normal_distribution"
#   is the mean and   is the standard deviation
normal_distribution = Normal( ,  )
# a is the upper bound and b is the lower bound; these can be set to +Inf or -Inf for ar
uniform_distribution = Uniform(a, b)
```

   There are lots of both univariate and multivariate distributions, as well as the ability to create your own, but we won't do anything too exotic here.

2. Draw samples. This uses the `rand()` command (which, when used without a distribution, just samples uniformly from the interval $[0, 1]$.) For example, to sample from our normal distribution above:

```
# draw n samples
rand(normal_distribution, n)
```

Putting this together, let's say that we wanted to simulate 100 six-sided dice rolls. We could use a Discrete Uniform distribution.

```
dice_dist = DiscreteUniform(1, 6) # can generate any integer between 1 and 6
dice_rolls = rand(dice_dist, 100) # simulate rolls
```

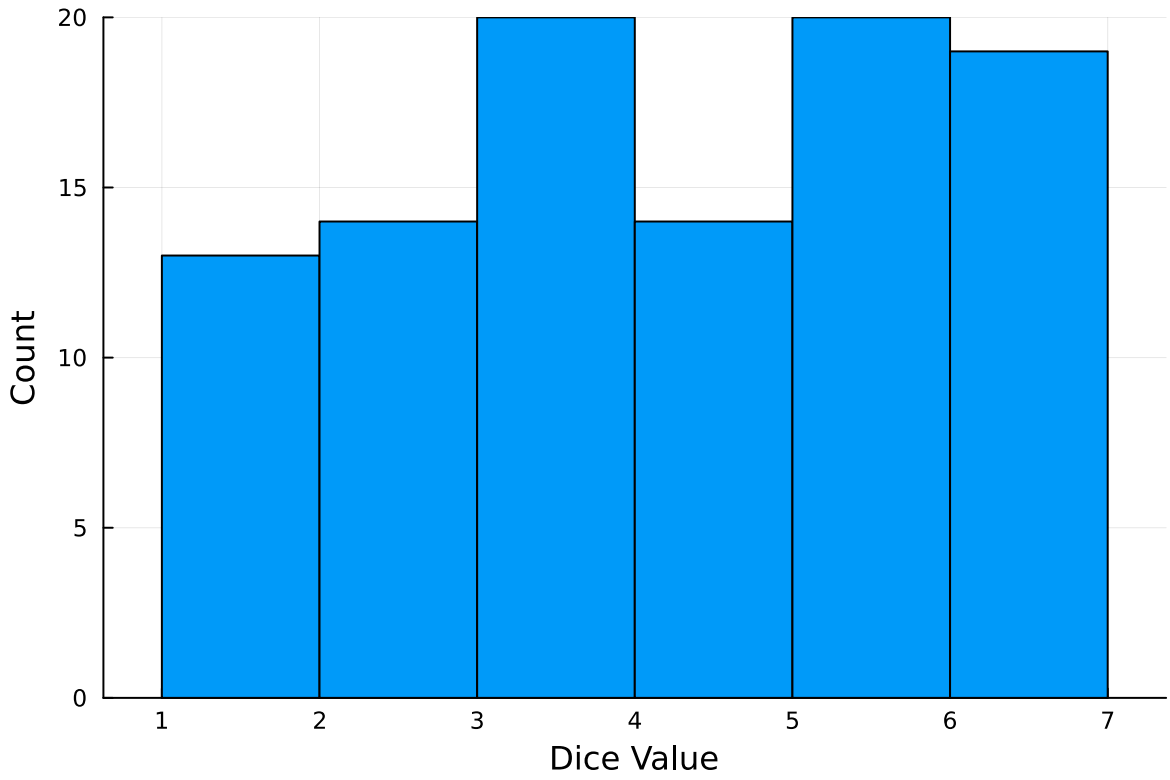
```
100-element Vector{Int64}:
 6
```

```
6
1
4
3
2
1
2
1
6

3
1
6
3
6
5
1
1
1
```

And then we can plot a histogram of these rolls:

```
histogram(dice_rolls, legend=:false, bins=6)
ylabel!("Count")
xlabel!("Dice Value")
```



Instructions

Remember to:

- Evaluate all of your code cells, in order (using a `Run All` command). This will make sure all output is visible and that the code cells were evaluated in the correct order.
- Tag each of the problems when you submit to Gradescope; a 10% penalty will be deducted if this is not done.

Problem

A common engineering problem is to quantify flood risk, which is typically computed by propagating a flood hazard distribution through a *depth-damage function* relating flood depths to economic damages. A reasonable depth-damage function for a house without a basement is a bounded logistic function,

$$d(h) = \mathbb{1}_{h>0} \frac{L}{1 + \exp(-k(h - h_0))},$$

where d is the damage as a percent of total structure value, h is the water depth (relative to the house's ground floor elevation) in m, $\mathbb{1}_{x>0}$ is the indicator function, L is the maximum loss in USD, k is the slope of the depth-damage relationship, and h_0 is the inflection point. We'll assume $L = \$200,000$, $k = 0.8$, and $h_0 = 3$.

For this problem, suppose that we have two different probability distributions characterizing annual maxima flood depths:

1. $h \sim LogNormal(1.2, 0.3)$;
2. $h \sim GeneralizedExtremeValue(3, 0.9, -0.15)$.

Plot histograms of samples from these distributions and conduct a Monte Carlo experiment for annual maximum flood damages using each. Answer the following questions:

- What are the Monte Carlo estimates of the expected value and the 99% quantile for the annual maximum damage the structure would suffer for each of these flood hazard distributions?
- How did you decide on your sample size?
- Why do you think the estimates differed or did not differ (you can also plot the depth-damage function to compare with the samples)?
- How might you decide (based on getting additional information, if needed) which distribution is "better"?

```
L = 200000 #maximum loss in dollars
k = 0.8 #slope of the depth-damage relationship
h0 = 3 #inflection point

hLog = LogNormal(1.2, 0.3) #h value for lognormal
hGen = GeneralizedExtremeValue(3, 0.9, -0.15) #generalized extreme value

function d(hLog) #d is a percentage
    if hLog > 0.0
        d = L / (1 + exp(-k*(hLog - h0)))
    else
        d = 0.0
    end
    return d
end

function d(hGen) #d is a percentage
    if hGen > 0.0
        d = L / (1 + exp(-k*(hGen - h0)))
    else
        d = 0.0
```

```
    end
    return d
end

histogram(d(hLog), legend=:false, bins=6)
ylabel!("Count")
xlabel!("Dice Value")
```

References

Put any consulted sources here, including classmates you worked with/who helped you.